Tool Retrieval under Distractor APIs with Memory-Based Recovery

Hemal Farouqe · Sheikh Mainul Islam · Tanjida Sultana Maria

Abstract

When LLM agents have to choose a tool from catalogues replete with near-identical descriptions, i.e. distractor APIs [1], [3], stateless selectors are prone to making the same error on like queries. To address this we have put a lightweight retrieve–select–execute loop in place that is backed by an experience memory [2]. This memory retains failed query–tool pairs and will down-rank those tools in keeping with query similarity.

We put the system to the test on a ToolAlpaca benchmark (380 APIs for 2,695 tools and 2,643 single-tool queries) using a fine-tuned all-MiniLM-L6-v2 encoder and cosine selector over five random API-level splits. The +10/+20/+30 mined distractors were included, as the 0-distractor case where the right tool is the only option is trivial. Distractors lead to a small drop in performance (baseline Top-1 of 0.674 goes to 0.662 between +10 and +30). But with similarity-weighted memory, first-attempt accuracy on well-worn errors at +30 is up from 0.425 to 0.551 (+12.6 pts; 374 pooled queries, positive in 5/5 splits, tool-cluster 95% CI [+7.8, +18.0]), barring one regression. The overall Top-1 sees a slight lift to 0.690 (CI [+0.3, +5.5]) from 0.662, though there is a 2.8 pt regression on queries for tools that have not yet failed, leaving the net gain marginal. A flat penalty yields a comparable familiar-error increase (+13.4 pts) but no overall headway (0.644), putting it 4.6 pts behind the similarity weighting.

As for final success in three attempts, memory makes little difference (0.881 vs. 0.878) and trims roughly 0.05 attempts per query. In a single-split run an LLM selector (GPT-OSS-20B, 0.811 baseline Top-1) derives no discernible advantage from the memory. Corrupting 20% of what is stored sends Top-1 below the no-memory mark, meaning non-semantic failures like timeouts or permission denials [4] need to be weeded out before they are put in storage.

Keywords: *Large language model (LLM) agents; tool retrieval; distractor APIs; experience memory; contrastive fine-tuning; failure recovery.*

1. Overview and Context

Among various steps an agent takes to respond to a user query, selecting the appropriate tool is one of the first steps that an agent must perform. However, in systems with many tools, it is challenging to select the correct tool from the list of possible tools. In practice there can be hundreds or thousands of possible APIs [1]. Often multiple tools will have very similar names and/or descriptions in natural language [1], [3] despite being used for different purposes. As a result, the agent can often select the wrong tool that seems appropriate based on its similarity in the embedding space.

Selecting the wrong tool is not trivial. This requires the system to run the tool, realise it has been incorrectly selected and attempt another tool. This leads to increased latencies and token usage. Additionally, the agent may continue to reason using the incorrect tool or perform an unexpected action [4]. Selecting the correct tool from the first attempt is therefore key to ensuring the agent performs reliably.

Suppose we have two tools named tool A (e.g. send_money_api: Transfers money to a recipient account) and tool B (e.g. transfer_money_friend_api: Checks progress of sending $50 to a friend). If their descriptions are very similar, when the query "Send $50 to my friend John" comes in, both tools would be retrieved. The agent would incorrectly select tool B, however tool A is the ground truth API. In a standard stateless retrieval system [1], this error would be forgotten immediately. If a similar query such as "Send $20 to my friend Sarah" comes into the system, the ranker would produce the same ranking and make the same error. Our core idea is to prevent the agent from making repeated errors by implementing a light memory for the agent [2]. When the agent makes an error and selects a tool that fails, this query and tool pair will be stored. This will cause the tool to be ranked lower in the future if a semantically similar query comes in.

Problem Statement. Given a user's question, a correct answer tool among a set of semantically confusing and incorrect tools, and previous failed attempts, the system should identify and choose the most appropriate answer tool to decrease the chance of choosing tools that have previous situations.

Objectives.

O1. Understand how additional (10, 20, 30) semantically similar distractor tools affect the tool retrieval and selection performance.

O2. Create an experience memory that records failed tool-query pairs and will penalise the ranking of tools for similar queries using a similarity weighted penalty for the query.

O3. Assess how much of an improvement there is with the experience memory for Recall@3, F1@3, Top-1 accuracy and familiar errors for both frozen and fine-tuned encoders and cosine and LLM selectors. Quantify the negative impacts on non-familiar queries and understand if introducing noise or corruption into the experience memory causes any issues with the tool selection.

2. Related Work and Background

2.1 Related Work

The four publications listed below inspired our research the most; they are all peer-reviewed articles published in 2026 that approach the problems of tool selection and agent reliability from unique perspectives. Kamiński et al. [1] empirically analyze the effect of environment scale on tool selection fidelity when growing an agent's pool of tools from the hundreds to the thousands. They suggest tool description synthesis as a method to reduce vector-space collisions between semantically similar distractor APIs by training to decompose the NLIT into "Retrieval–Plan–Select" subtasks. Benchmarked with Recall@k and F1@k metrics under degraded system conditions with both random and semantically sorted distractors, [1] is the backbone of our primary baseline pipeline, but their system only operates on queries one at a time and does not learn from its mistakes. Cheng et al. [2] bridges the gap of learning from failure via Mem²Evolve, a hierarchical memory framework that implements an "Experience Memory" component responsible for storing strategic knowledge extracted from past agent trajectories—both successful and failed—and instructing dynamic retrieval of tools/extensions capable of overcoming previously missed subgoals. Inspired by [2], our method utilizes memory to store information relevant to the problem of look-alike distractors, but does not learn at the level of whole capabilities or trajectory-based subgoal planning. Zappia et al. [3] move away from cloud-based systems to quantify error trends of heterogeneous low-power agentic frameworks deployed on edge devices when retrieving the correct tool API from a large pool of available tools under distractor pressure; they find that

specifically, an LLM's likelihood of calling the correct tool decreases significantly when available tools that are not part of the ground-truth plan share tokens with the intended function name. Lastly, Singh and Madisetti [4] introduce MCP-Secure, a general-purpose runtime layer that provides access-control for the Model Context Protocol (MCP) architecture by keeping state over the course of a session in a lightweight finite state machine and mediating all tool calls to ensure agents adhere to least-privilege expectations without altering the behavior of the LLM itself. While [4] is not concerned with the semantics of tool choice like our research is central to our robustness analysis because in real deployments tool calls often fail due to permission denials or timeouts rather than retrieval errors (Sections 5.8 and 6).

Work	Main focus	Contribution to our setting	Gap relative to our problem
[1] Kamiński et al.	Tool selection and collision mitigation at scale	Baseline, Recall/F1 protocol, distractor stress tests	Stateless, no learning from failures
[2] Cheng et al.	Experience distillation in self-evolving agents	Motivates memory of past experience	Not applied to distractor-driven retrieval errors
[3] Zappia et al.	Distributed edge agents and error patterns	Evidence that token/tool similarity hurts calling accuracy	Not focused on retrieval recovery
[4] Singh and Madisetti	Runtime access control (MCP-Secure)	Defines how tools are executed under policy	Does not assess whether the selected tool is semantically correct

Research gap. Scalable tool retrieval [1], experience memory [2], and tool-calling error and access governance [3], [4] have all been treated as independent problems in existing works we reviewed. We focus on their intersection: leveraging failures from an agent's own retrieval attempts to down rank incorrect tool calls made because of semantically similar distractors, and retrieve the right API on later tries.

2.2 Background

Given a query, a tool-using agent would first generate a shortlist of candidate tools to use by encoding the query and retrieving most similar tools by cosine distance against encoded tool descriptions in vector space. It will then have a selector (either highest cosine score or an LLM) choose which tool to use from that shortlist. We evaluate shortlist retrieval using Recall@k and F1@k metrics [1]. Recall@k measures if the ground-truth tool is in the top k results. F1@k is the harmonic mean of Precision@k and Recall@k [1]. As you add more semantically similar distractor tools, they begin vying for those top k spots and crowd out the correct tool.

3. Methodology

3.1 Pipeline

Our system follows an end-to-end retrieve-select-execute pipeline (Fig. 1). A user task will enter the pipeline and match with the tools within the tool repository. The retriever will return a ranked list of

candidate tools. This contains the ground-truth tool plus several semantically similar distractor tools. The selector will pick one tool from the list and execute it. If the tool executed successfully (matches the ground-truth API), then our system will return the final results. However, the pairings of failed queries and tools will be added to the experience memory, and the re-ranking of tools will be updated based on these failures. The system will then attempt to find a successful tool again with a maximum number of R (R=3) attempts. Tools that have been tried within the current episode will be hard-excluded.3. Methodology.

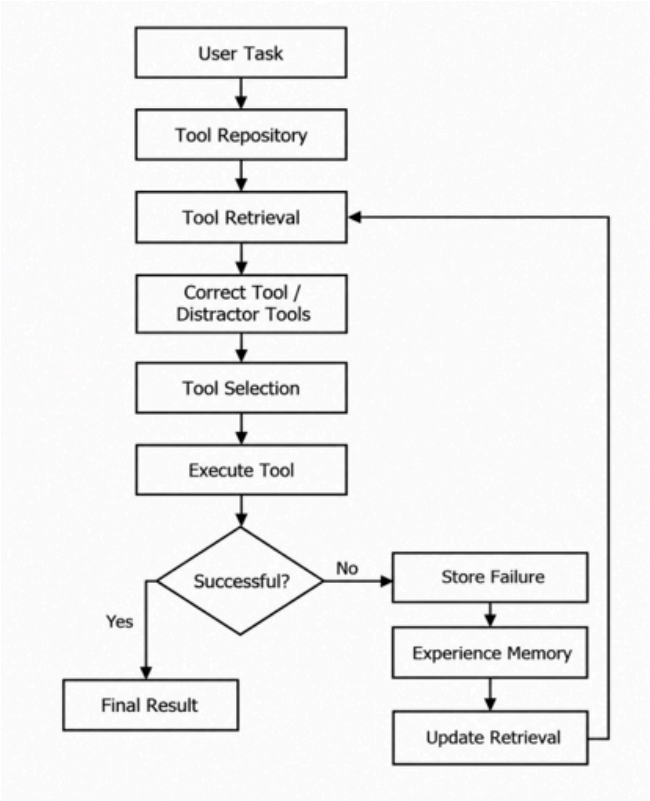


Fig. 1. Proposed retrieve–select–execute loop with failure memory.

3.2 Retrieval and memory-based re-ranking

Given a sentence embedding encoder f (all-MiniLM-L6-v2), we calculate the basic retrieval scores for each tool t as the cosine similarity between each query q and tool t: s(q, t) = cos(f (q), f (t)). To store previous failed pairs (q i,t i) we use an experience memory M. We then calculate the penalty factor for the tool t for the query q using the similarity between the most similar previous query where the tool t failed, weighted by the similarity threshold τ. Finally, we apply the penalty to the score as follows:

$$\pi(q, t) = \max \cos(f(q), f(q_i)) \text{ over all } (q_i, t) \text{ in } M, \text{ set to 0 if below } \tau$$

$$s'(q, t) = s(q, t) - \lambda\pi(q, t)$$

We weight the penalty by the similarity between queries. We want tools that are not suitable for a particular task to be penalised but not suppressed for unrelated tasks. We do not want penalties to affect a true ground truth tool for a different task. We calculate hyperparameters ($\lambda = 0.3$, $\tau = 0.6$) using a grid search over $\lambda \in [0.1, 0.3, 0.5]$ and $\tau \in [0.6, 0.7, 0.8]$ using the seed 42 validation split with +30 distractors (See Section 5.1). These values are then fixed for all five evaluation splits.

3.3 Selection

We fix the task instruction, tool description, and selector prompt across all settings. Therefore, any variation in performance must be due to differences in retrieval, encoder training, and experience memory. By default, the selector uses a deterministic top-1 selection based on (penalized) cosine scores s′(q,t). In Section 5.7, we evaluate using an LLM-based selector (openai/gpt-oss-20B at temperature 0.0 using Groq), which selects from the top-5 shortlisted tools ranked by s ′ (q,t) and returns the index of its selected tool.

3.4 Dataset

Our controlled benchmark was constructed from the ToolAlpaca dataset (train_data.json). For each API the functions were considered as individual tools (function_name: short_description), resulting in a total of 2,695 tools. All queries in the dataset that call a single ground truth function were included in the dataset. This resulted in a total of 2,643 queries across 380 APIs. We then include multi-tool or empty examples (1,612). We perform 5 random API splits (42, 1, 2, 3, 4) to measure the variance that can occur when splitting the familiar error sub-set. The main cosine selector experiments were performed on each of these API splits. 70% of the APIs were in the training set, 15% in the validation set and 15% in the unseen test set. For example, split 42 had 1,874 queries in the training set (1,599 tools), 375 queries in the validation set (330 tools) and 394 queries in the test set (334 tools). The two-phase memory approach produced a total of n=1,023 test examples when pooling across all five splits. Since the same 380 APIs were randomly split between each split, there will be overlapping test sets. Hard semantic distractors were obtained from the ToolAlpaca dataset. For each correct tool, all the tools in that split were encoded using the frozen base encoder (all-MiniLM-L6-v2). Then the top 30 similar tools based on cosine similarity were taken as the distractors. We then tested with +10, +20, +30 distractors (excluding 0 distractors). The performance was measured by whether the correct tool was chosen. If not, it was recorded as a failure.

We tested using two different encoders, the all-MiniLM-L6-v2 model. One was the original frozen pre-training version and the other was fine-tuned using the training split for each seed. This was done for 3 epochs (batch_size=32, learning_rate= 2×10^{-5}, warmup_ratio=0.1, using fp16 and no-duplicate batch sampling). The model was trained using a contrastive loss (MultipleNegativesRankingLoss), where 3 hard negative mined distractors were used from the top 10 nearest neighbours:

$$L = -\log [\ \exp(s(q, t^+)/\theta) / (\ \exp(s(q, t^+)/\theta) + \Sigma_{t^-} \exp(s(q, t^-)/\theta)\ )\ ]$$

Where t^+ is the ground truth tool and t^- is the distractors. Θ is the temperature. This allowed us to measure how much of an improvement was made by using a contrastive representation learning method rather than memory.

3.5 Evaluation metrics

We follow [1] and use Recall@3 and F1@3 on the initial ranking after memory penalty. We also report overall Top-1 accuracy (initial accuracy on all test queries), familiar error Top-1 accuracy (initial Accuracy on test queries where ground-truth tool failed on its warmup query), unfamiliar Top-1 accuracy (initial accuracy on test queries where ground-truth tool succeeded on its warmup query), multi-attempt recovery rate (success after R=3 attempts) and average attempts per query. For comparison across the 5-fold pooling we carry out an exact paired McNemar test. We also perform a bootstrapped 95% confidence interval (CI) using 10,000 resamples with tool clusters. This treats ground-truth tools as clusters so that correlated queries from the same tool (even if they are in overlapping test splits) are

treated as one cluster. As the McNemar test does not take into account this clustering at a tool level, we will use the 95% CI as our main inference criteria.

4. Baseline and Extensions

Baseline. We employ stateless embedding-based retrieval, with top-1 cosine or LLM selection. Evaluation is done using the Recall/F1 protocol and distractor stress tests from [1]. Our retry-only baseline (for up to R=3 attempts) does not remember tools attempted during the current query, only hard-excluding those that have been previously used. Hence, if two tools A and B have very similar descriptions, the agent may confuse them at some point; this confusion will then happen again for subsequent queries.

Extensions. Each of these extensions are designed to address the problem of the stateless system:

E1. Store the failed query/tool pairs in memory M. If a query has previously resulted in a failure then the tool gets a flat penalty (λ=0.3) for all queries.

E2. Multiplying the penalty by the similarity between queries, this similarity is calculated using $\pi(q,t)$ and only applies when the similarity is greater than τ (0.6) (Section 3.2). The similarity will result in down-ranking the tool for similar queries but a lower penalty for unrelated queries.

E3. If a query has previously caused a failure then the tool is excluded from the short list.

E4. We evaluate our systems with all of the configurations using +10, +20 and +30 mined distractors. This evaluates how well the memory performs as more semantic confusion is added.

Ablation and failure analysis design. We compare Baseline (retry only), E1 only, and E1+E2 strategies across 5 different random API-level train/test splits. We also evaluate E1+E2+E3 (E3 never triggers in one-shot warm up). Λ and τ are tuned using the seed=42 validation split (R=3). We also vary the proportion of corrupted stored failures (ranging 0-60%) in +30 distractors for seed 42 to quantify memory pollution.

Configuration	Memory	Context penalty	Filtering	Distractor count
Retry-only baseline	No	No	No	+10 / +20 / +30
E1	Yes	No	No	+10 / +20 / +30
E1+E2	Yes	Yes	No	+10 / +20 / +30
E1+E2+E3 (untriggered)	Yes	Yes	Yes (sim $\geq$ 0.85)	+10 / +20 / +30

5. Results and Analysis

5.1 Experimental Setup

Data and multi-split. The benchmark was created using data from ToolAlpaca (2,695 tools, and 2,643 single-tool queries over 380 APIs). To perform the main experiment with the fine-tuned encoder and cosine selector, we randomly split the data 5 times at API level (seeds 42, 1, 2, 3 and 4). This gives 5 different splits of the 380 APIs into 70% train, 15% validation and 15% test. Each train set is used to

fine-tune a different all-MiniLM-L6-v2 encoder. Due to the fact that the splits all contain the same 380 APIs, there will be overlap between the test sets. The main comparison with the frozen encoder (Section 5.6), LLM selector (Section 5.7), memory pollution (Section 5.8) and Fig. 2 all use the main single split (seed 42).

Distractors. Distractors were not generated but extracted from ToolAlpaca. For each correct tool, the 30 most similar tools (based on base encoder cosine similarity) in the same split make up the distractors. We evaluate with +10, +20 and +30 distractors. With 0 distractors the task is trivial, as there is only one candidate, so this is excluded.

Memory protocol. For each test-split with ≥ 2 queries, during phase 1 the first query was answered without using memory, with up to R=3 attempts to answer, and tools that were attempted within the current episode excluded from future attempts. This built the memory M based on any incorrect tool selection made. During phase 2, remaining queries were answered with memory locked. This resulted in a total of 1,023 test queries from the 5 splits. Of these, there were 367 (372, 374) queries from +10 (+20, +30) distractors where the ground truth tool had failed on their first attempt (i.e. in the 'Familiar error' subset). The remaining queries made up the 'Non-familiar' subset, where the ground truth tools had succeeded on their warm-up query (656, 651, and 649 queries from +10 (+20, +30)).

Hyperparameter sweep and statistical testing. A grid search was carried out across the 30 distractor validation split using seed-42 (41 queries in the 'Familiar' subset). This identified a flat landscape when varying λ (0.1, 0.3, 0.5) and τ (0.6, 0.7, 0.8). The best performing hyperparameters were chosen (λ=0.3, τ=0.6) achieving a Top-1 score of 0.610 and 0.791 on the 'Familiar' subset and overall respectively. These hyperparameters were then used for all 5 test splits. The difference between the 5 test splits was calculated using paired analysis. 95% confidence intervals (CIs) were calculated using bootstrap sampling (10,000 samples) of tool clusters (grouped by ground truth tool). McNemar tests were also performed to calculate exact p-values. However, due to the clustered nature of our data the tool cluster CI was mostly used. The McNemar test does not account for correlation between queries that have the same tool.

Metrics. Top-1 is the accuracy of answering all test queries on the first attempt (1,023 queries). Familiar Top-1 is the accuracy of answering queries in the 'Familiar error' subset on the first attempt. Success ≤ 3 is the fraction of queries that can be solved within 3 attempts (R=3), where previously attempted tools are excluded. Avg. attempts is the average number of attempts required to answer each query (unanswered queries count as 3 attempts). On the seed-42 split, the Recall@3 metric was also recorded. As every query has exactly one ground truth tool, Precision@3 = Recall@3 / 3. This means the F1@3 metric is simply a deterministic linear scaling of Recall@3 (F1@3 = 0.5 Recall@3).

5.2 Effect of Distractors and Overall Memory Trade-off (H1, H2)

Table 1. Pooled 5-split test results (fine-tuned encoder, cosine selector; n = 1,023 overall queries, n = 367 / 372 / 374 familiar-error queries at +10 / +20 / +30 distractors).

Distractors	Config	Overall Top-1	Familiar Top-1	Success ≤ 3	Avg. attempts
+10	Retry-only baseline	0.674	0.439	0.920	1.490
+10	E1	0.683	0.602	–	–
+10	E1+E2	0.710	0.580	0.912	1.437

Distractors	Config	Overall Top-1	Familiar Top-1	Success ≤ 3	Avg. attempts
+20	Retry-only baseline	0.666	0.435	0.896	1.520
+20	E1	0.663	0.583	–	–
+20	E1+E2	0.698	0.573	0.890	1.472
+30	Retry-only baseline	0.662	0.425	0.881	1.536
+30	E1	0.644	0.559	–	–
+30	E1+E2	0.690	0.551	0.878	1.490

H1 (distractors hurt) is supported, though the decline is modest. Increasing the distractor pool size from +10 to +30 reduces the retry-only baseline by 0.012 in Top-1 overall accuracy (0.674 to 0.662), 0.014 in Top-1 familiar error accuracy (0.439 to 0.425) and 0.039 in 3-attempt recovery accuracy (Success ≤ 3) (0.920 to 0.881), while increasing the average number of attempts (1.490 to 1.536). For the shortlist evaluation on the single split (seed 42), we also see a decrease in baseline Recall@3 (0.882 to 0.867 and 0.846 for +10, +20 and +30 respectively), with F1@3 scores of 0.441, 0.433 and 0.423. This decrease is gradual, as the most difficult nearest neighbour collisions are already included within the top-10 mined distractor set.

H2 (memory lifts overall accuracy, shortlist recall, and final recovery) reveals a genuine trade-off between familiar and non-familiar queries. Similarity-weighted memory (E1+E2) increases overall Top-1 accuracy slightly ($p < 0.001$). On the pooled results across 5 splits, memory increases Top-1 accuracy by 3.5% (+10: 0.674 to 0.710, tool-cluster 95% CI [+0.009, +0.063], helped 53, hurt 17), 3.2% (+20: 0.666 to 0.698, CI [+0.005, +0.060], helped 52, hurt 19), and 2.8% (+30: 0.662 to 0.690, CI [+0.003, +0.055], helped 48, hurt 19). Despite a positive change in overall Top-1 accuracy for all 5 splits, the low-end of the tool-cluster 95% CI is just above zero at +30 (0.003). This means there is a very slight overall increase in accuracy.

The results show that there is a significant negative impact on non-familiar queries The increase in overall accuracy is small due to **memory negatively impacting performance on non-familiar queries ($p < 0.001$)**. Where the ground truth tool did not fail in phase 1 (656 queries at +10, 651 queries at +20, 649 queries at +30), memory helps 0 queries and hurts 16-18 queries. Top-1 accuracy decreases by 2.4% (+10: 0.806 to 0.782, diff -0.024, tool-cluster 95% CI [-0.045, -0.008]), 2.8% (+20: 0.797 to 0.770, diff -0.028, CI [-0.050, -0.010]) and 2.8% (+30: 0.798 to 0.770, diff -0.028, CI [-0.050, -0.010]). This occurs because a legitimate tool T was incorrectly retrieved as a distractor whilst warming up another tool. T is stored in M. Later when T is actually required for a test query, where T has a cosine similarity of $\geq \tau$ (0.6) to the previous query, it will be down-ranked. The overall increase in Top-1 accuracy is small due to the increase in familiar error accuracy (+12.6%) being much larger than the decrease in non-familiar accuracy (-2.8%). This is because there were less queries in the familiar error group than the non-familiar group (374 queries at +30). However, there is no improvement in Recall@3 when using E1+E2 due to the fact that memory only down-ranks queries. This can be seen in the shortlist evaluation on the single split (seed 42), with the scores decreasing further (0.872, 0.856, 0.836) compared to baseline (0.882, 0.867, 0.846).

5.3 Recovery on Familiar Errors (H3)

In terms of the familiar error subset, similarity-based memory (E1+E2) gives a substantial benefit across all levels of distractors and splits (Table 1). Overall, similarity-based memory (E1+E2) improves accuracy by increasing Top-1 accuracy from 0.439 to 0.580 (+10) for familiar errors (+**14.2 points**, 95% CI [+0.092, +0.200] for all splits combined, 53 tools improved, 1 tools worsened, $p < 0.001$). The average improvement per split was +0.140 (±0.049). From 0.435 to 0.573 (+20) (+**13.7 points**, 95% CI [+0.087, +0.194], 52 tools improved, 1 tools worsened, $p < 0.001$). The average improvement per split was +0.135 (±0.045). From 0.425 to 0.551 (+30) (+**12.6 points**, 95% CI [+0.078, +0.180], 48 tools improved, 1 tools worsened, $p < 0.001$). The average improvement per split was +0.125 (±0.041). Across all 374 familiar errors (+30), similarity-based memory (E1+E2) fixes 48 baseline errors but only creates 1 regression. Overall, +30 accuracy improved for 5/5 splits. For example, +30 Top-1 familiar error accuracy increased from 0.397 to 0.534 (seed 42), 0.418 to 0.481 (seed 1), 0.588 to 0.691 (seed 2), 0.368 to 0.529 (seed 3) and 0.378 to 0.537 (seed 4). This provides good evidence to support H3 for the cosine selector.

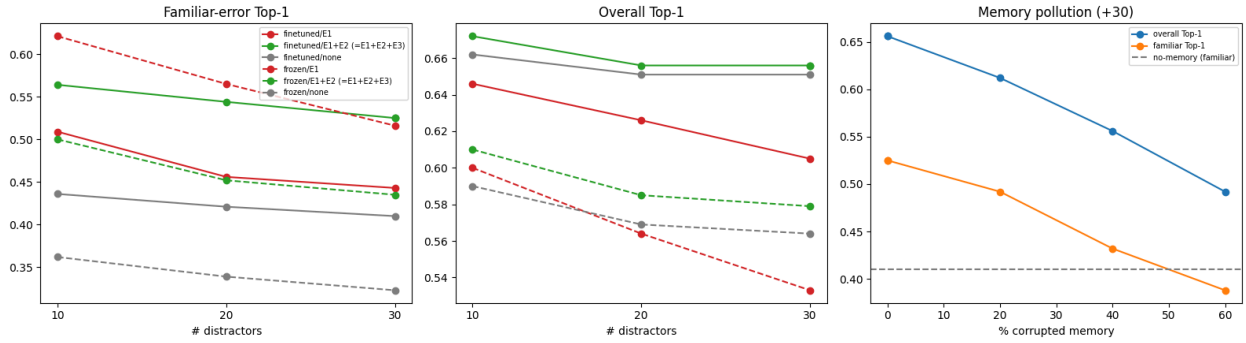


Fig. 2. Single-split (seed 42) evaluation results (results.png). Left and middle panels: Familiar-error Top-1 and Overall Top-1 across +10, +20, and +30 distractors for fine-tuned and frozen encoders on seed 42 (matching Table 2; E1+E2+E3 is identical to E1+E2). Right panel: Overall and familiar-error Top-1 under 0%–60% memory pollution at +30 distractors on seed 42 (matching Table 4).

5.4 Ablation: E1, E2 and E3

E1 (flat penalty) is overall-neutral and causes heavy regressions on unrelated queries. Using a flat (ungated) penalty (λ=0.3) on all failed tools in memory, we achieve similar improvements on Top-1 recall for familiar error questions as E1+E2. Using flat penalties improves Top-1 by 16.3 (0.439 → 0.602, 86 improved / 26 worse, $p < 0.001$), 14.8 (0.435 → 0.583, 77 improved / 22 worse, $p < 0.001$), and 13.4 (0.425 → 0.559, 71 improved / 21 worse, $p < 0.001$), for +10, +20, and +30 respectively. The problem with this is that E1 penalises all previous failed tools, regardless of the query's context. This reduces performance across all queries, with 87, 91, and 99 queries worsening. This means there is no significant difference between the baseline and E1 for overall Top-1 performance. E1 improved Top-1 by 0.9 (0.683 vs. 0.674, 95% CI tool cluster [-0.035, +0.051], $p = 0.55$), reduced Top-1 by 0.3 (0.663 vs. 0.666, 95% CI tool cluster [-0.047, +0.039], $p = 0.88$), and reduced Top-1 by 1.8 (0.644 vs. 0.662, 95% CI tool cluster [-0.062, +0.025], $p = 0.21$), for +10, +20, and +30 respectively.

E2 (similarity-weighted penalty) beats E1 overall by sharply reducing regressions, not by increasing familiar-error gains. Using a similarity weighted penalty gives a Top-1 difference of -2.2 (0.580 vs. 0.602, 95% CI tool cluster [-0.096, +0.053], 31 improved / 39 worse, $p = 0.40$), -1.1 (0.573 vs. 0.583,

95% CI tool cluster [-0.072, +0.059], 27 improved / 31 worse, p = 0.69), and -0.8 (0.551 vs. 0.559, 95% CI tool cluster [-0.066, +0.061], 26 improved / 29 worse, p = 0.79), for +10, +20, and +30. Using E2 prevents regression on other questions. Across all 1,023 queries E1+E2 only regresses 17-19 queries, whereas E1 regressed 87-99. Only 1 of these were familiar queries. Compared to the E1 this gives a much better performance on overall Top-1. E1+E2 performs much better than E1 improving Top-1 by 4.6 (0.690 vs. 0.644, 95% CI tool cluster [+0.014, +0.080], 86 improved / 39 worse, p < 0.001), 3.5 (0.698 vs. 0.663, 95% CI tool cluster [+0.004, +0.067], 78 improved / 42 worse, p = 0.001), and +2.6 (0.710 vs. 0.683, 76 improved / 49 worse), for +30, +20, and +10. The p value (0.020) using McNemar with no clustering shows significant differences for +10. However the 95% CI tool cluster is [-0.006, +0.059] meaning we cannot clearly differentiate E1+E2 and E1.

E3 (filtering at similarity ≥ 0.85) never triggered and remains untested. Our phase 1 protocol only allows one warm-up query per tool. Due to this and the fact that queries in ToolAlpaca are almost always unique in natural language, all settings never activated E3. As a result, E1+E2+E3 is identical to E1+E2. This means we have not included individual rows for E1+E2+E3 in the results. Therefore we have not disproven this method.

5.5 Is the Gain From Memory or From Retrying?

The baselines (Retry only) and E1+E2 from our overall 5-way split results (Table 1) also exclude all tools that have been attempted in the current query retry loop (up to R=3 retries) to ensure a fair like-for-like comparison. **Experience memory provides no benefit to final success when using 3 retries.** Using the overall results for Success ≤ 3, both E1+E2 and the baseline have very similar success rates with no significant difference between them (+10; E1+E2: 0.912 vs. Retry only: 0.920 (d = -0.008, 95% CI [tool -0.019, +0.002], 5 helpful, 13 harmful; p=0.096), +20; E1+E2: 0.890 vs. Retry only: 0.896 (d = -0.007, 95% CI [tool -0.019, +0.003], 6 helpful, 13 harmful; p=0.17), +30; E1+E2: 0.878 vs. Retry only: 0.881 (d = -0.003, 95% CI [tool -0.016, +0.010], 10 helpful, 13 harmful; p=0.68). This is because when the highest ranking distractor fails in the first attempt, due to the hard exclusion, it is excluded on the second attempt like if there had been a penalty. However, experience memory can be useful in **allowing correct choices on the first attempt in known errors** (Section 5.3). This reduces the average number of tools attempted per query by approximately 0.05 across all distractor levels (+10; 1.490 → 1.437, +20; 1.520 → 1.472, +30; 1.536 → 1.490). This reduces the number of failed attempts at using a tool.

5.6 Frozen vs. Fine-tuned Encoder (Single Split, Seed 42)

Table 2. Comparison between frozen and fine-tuned versions of all-MiniLM-L6-v2 encoder for single-split (seed 42) evaluation with cosine selector. 195 test queries. 55/61 familiar error queries for fine-tuned and 58/62 for frozen (at +10/+30).

Distractors	Config	Frozen Top-1	Frozen Fam.	Fine-tuned Top-1	Fine-tuned Fam.
+10	Baseline	0.590	0.362	0.662	0.436
+10	E1	0.600	0.621	0.646	0.509
+10	E1+E2	0.610	0.500	0.672	0.564
+30	Baseline	0.564	0.323	0.651	0.410

Distractors	Config	Frozen Top-1	Frozen Fam.	Fine-tuned Top-1	Fine-tuned Fam.
+30	E1	0.533	0.516	0.605	0.443
+30	E1+E2	0.579	0.435	0.656	0.525

Contrastive InfoNCE fine-tuning improves the baseline overall Top-1 performance by between 7.2 and 8.7 (0.590 → 0.662 (+10); 0.564 → 0.651 (+30)) on the single-split (seed 42) setting (Table 2 and Figure 2, left and centre plots). Incorporating experience memory (E1+E2) provides an improvement of 11.2-13.8 in Top-1 performance on familiar errors, on top of either encoder (11.2 improvement on the frozen encoder (0.323 → 0.435); 11.5 improvement on the fine-tuned encoder (0.410 → 0.525) (+30)). This indicates contrastive training of the encoder and failure memory at test time are largely complementary mechanisms. The fine-tuned results for single-split (seed 42) shown in Table 2 and Figure 2 are slightly different to those seen in the seed 42 row of our 5-split run (0.651 (baseline) and 0.410 (familiar) for fine-tuned vs. 0.646 and 0.397 respectively). This is due to the lack of bit deterministic training when using mixed-precision training (fp16) on the GPU. The one anomaly with the frozen encoder results is that using flat penalty (E1) provides a better result on familiar errors (0.516 (+30) vs. 0.435 (+30)) but results in a worse overall Top-1 (0.533 vs. 0.579). This is because the hyperparameters λ=0.3 and τ=0.6 were optimised for the fine-tuned encoder and were not optimised for the similarity distribution of the frozen encoder.

5.7 LLM Selector (Single Split, Seed 42)

We ran the evaluation again on the seed-42 test split with +30 distractors and used an LLM (GPT-OSS-20B via Groq, temperature=0) to select the response. The top 5 ranking responses based on penalized cosine scores are shown to the LLM selector and the LLM returns a count. We exclude any queries that failed during API calls across the 107 multi-query test tools (191 test queries). See Table 3.

Table 3. LLM selector evaluation on the single-split (seed 42) test set (+30 distractors).

Config	n	Top-1	Familiar Top-1 (n = 34)	Success within R=3	Avg. attempts	API errors
Baseline	180	0.811	0.500	0.972	1.233	11
E1+E2	179	0.816	0.529	0.972	1.229	12

Two findings stand out.

The LLM selector performs significantly better than the top-1 cosine selector. The baseline Top-1 increases from 0.651 (seed-42 cosine) to 0.811 and the familiar-error Top-1 increases from 0.410 to 0.500 as the LLM considers all of the tool details in its selection over a 5 tool shortlist.

The LLM has no measurable impact when using memory. The overall Top-1 only increases from 0.811 to 0.816. The familiar-error increases from 0.500 to 0.529 which is only one query out of 34. This means we can't claim H2 or H3 are true for the LLM selector. Since the penalty just re-orders a list of 5 tools and the ground truth tool is often already in the top-5, a good LLM will always select it. Λ and τ were also not re-tuned for the LLM.

5.8 Vulnerability to Memory Pollution (Single Split, Seed 42)

For the seed-42 condition with +30 distractors, we introduced some corruption to the memory of E1+E2 where the correct tool was marked as failed. This is a very severe form of noise simulating infrastructure or permission errors.

Table 4. Memory pollution evaluation on the single-split (seed 42) test set (+30 distractors, E1+E2, n = 195).

Corrupted memory	Top-1	Familiar Top-1	Recall@3	F1@3	Avg. attempts
0%	0.656	0.525	0.836	0.418	1.549
20%	0.612	0.492	0.817	0.409	1.634
40%	0.556	0.432	0.790	0.395	1.747
60%	0.492	0.388	0.761	0.380	1.868

The accuracy degrades as corruption increases. From 20% corruption (0.612), the overall Top-1 accuracy is lower than the no-memory case (0.651) using one split. The familiar error Top-1 accuracy remains higher than the no-memory case (0.410) up to around 40% (0.432) but drops below this value at 60% (0.388). This crossing point occurs at around 50% corruption. This demonstrates that in order to have experienced memory we require a clean failure signal. In practice, there will be non-semantic failures such as timeouts or permission denials [4] which need filtering before adding them to memory.

5.9 Summary and Limitations

Hypothesis	Result
H1: distractors reduce baseline performance	Supported (modest decline: pooled Top-1 0.674 → 0.662, Success ≤ 3 0.920 → 0.881 from +10 to +30)
H2: memory lifts Recall@k, F1@k, overall Top-1, and recovery	Partly supported: small overall Top-1 gain (+2.8 to +3.5 pts; +30 CI [+0.3, +5.5]), offset by a 2.4–2.8 pt regression on non-familiar queries; Recall@3/F1@3 not improved; 3-attempt recovery unchanged vs. retry-only (0.878 vs. 0.881)
H3: memory curbs repeated errors on familiar tools	Solidly supported with the cosine selector (+12.6 to +14.2 pts pooled; +30 CI [+7.8, +18.0], positive in 5/5 splits); not supported with the LLM selector
Pollution risk	Confirmed (single-split overall Top-1 drops below no-memory baseline at 20% corruption: 0.612 vs. 0.651)

Limitations. (i) Our distractors come from existing functions implemented within ToolAlpaca, hence are less extreme than paraphrased distractors. (ii) The experience memory creates a true trade off in the Results. Lowering the rank of tools that fail during warm up decreases the accuracy of non familiar Queries by 2.4 to 2.8. This results in a very small overall Top-1 improvement (+2.8). While there are 5

Random splits at API level (1,023 test queries), the tool clusters are bootstrapped meaning that the Test queries overlap between the 5 splits. This means that there is a very small improvement between each Split in the overall Top-1 results (+0.027±0.022 @+30). The n=5 splits are on the edge of significance. The frozen encoder, LLM selector and memory pollution experiments only used one split (seed 42). (iii) The hyperparameters were tuned on the seed 42 fine-tuned cosine setup (λ=0.3, τ=0.6). This was then applied without re-tuning to the remaining models. Approximately 6% of LLM calls were excluded due To API errors. (iv) E3 was never activated throughout the single query warm up. Hard filtering has not been tested. (v) E3 was never triggered when using a single query warmup. Hard filtering therefore was not tested.

6. Discussion and Future Directions

Challenges and empirical lessons. Firstly, creating natural distractors requires care to avoid synthetic paraphrase distractors being easily distinguishable or unrealistic, which would artificially increase memory effectiveness. By utilising nearest neighbour methods from the ToolAlpaca [1] library, we were able to avoid this issue and create real semantic collisions [3]. Secondly, there are many reasons that a tool will fail in practice, with network timeout, rate limiting (11-12 Groq API calls failed due to this issue, see Section 5.7), and run-time access restrictions [4] being common issues. We demonstrate in Section 5.8, by introducing non-semantic errors into our memory, that the performance of Top-1 decreases below the stateless model. Even with only 20% of queries being corrupted, the performance is reduced and highlights that in production, it is important to differentiate between policy and infrastructure errors. Thirdly, the addition of the experience memory creates the trade-off between correcting known errors, whilst ensuring that we aren't penalising tools unfairly. In Section 5.4, we show that using an ungated flat penalty (E1) and a similarity penalty (E1+E2) give a similar increase to known errors (+13.4 vs +12.6 at +30). However, E1 has 87-99 regressions in royal, resulting in no overall improvement (+0.644 vs +0.662 at +30). Using a similarity penalty reduces the amount of regressions by 5 times (17-19 queries regressed in total) and performs better than E1 (+4.6 total at +30). However, there is still a significant impact on non-known queries where the ground truth tool was previously selected incorrectly as a distractor, with a 2.4-2.8 point regression. Lastly, whilst using a cosine similarity based tool selector does provide significant improvements to first attempt known errors [2], excluding retries within an episode gives the same performance (0.881 vs 0.878). A good LLM based tool selector can resolve most ambiguity from the top 5 shortlisted tools (0.811 Top-1) and doesn't benefit from having score based memory.

Next steps.

Competitor-conditioned memory penalties to eliminate non-familiar regressions: condition the Memory penalty on the co-occurrence of the competing tool (or store directional pairs indicating which tool succeeded after a failure) so that a legitimate tool is not penalized when serving as the ground-truth API on its own queries.

Multi-query streaming memory to test E3 filtering: extend the memory-building protocol beyond a Single warm-up query per tool to multi-episode streams so that repeated high-similarity failures ($\cos(f(q), f(q_i)) \geq 0.85$) accumulate and actively test hard shortlist filtering (E3).

Runtime failure filtering with access control: integrate the experience memory with a runtime Access-control layer such as MCP-Secure [4] so that permission denials, rate limits, and timeouts are filtered out before updating failure memory.

7. Team Contribution

Member	Contribution
Hemal Farouqe	Overview and Context; Baseline and Extensions; Benchmark Data Pipeline
Sheikh Mainul Islam	Methodology; Encoder Fine-Tuning and Memory Implementation; Future Directions
Tanjida Sultana Maria	Related Work and Background; Results and Analysis; Multi-Split Paired Evaluation and Ablation; Discussion

References

[1] J. Kamiński, I. Galyukshev, S. Chuprin, A. Kuznetsov, and A. Kalyuzhnaya, "Too many tools, too much confusion? Navigating agentic tool selection at scale," *Algorithms*, vol. 19, no. 6, p. 447, 2026, doi: 10.3390/a19060447.

[2] Z. Cheng, Z. Liu, Y. Shan, X. Wang, X. Zhu, Y. Ma, H. Wang, Y. Guo, W. Lin, and Y. Wang, "Mem²Evolve: Towards self-evolving agents via co-evolutionary capability expansion and experience distillation," in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2026, pp. 20784–20831, doi: 10.18653/v1/2026.acl-long.952.

[3] L. L. Zappia, M. Lattuada, M. Bianchessi, and D. P. Pau, "Heterogeneous low-power agentic architecture for edge distributed home surveillance," *Electronics*, vol. 15, no. 14, p. 3175, 2026, doi: 10.3390/electronics15143175.

[4] G. Singh and V. K. Madisetti, "MCP-Secure: A runtime access control layer for privilege-aware LLM agent tooling," *IEEE Open J. Comput. Soc.*, vol. 7, pp. 504–513, 2026, doi: 10.1109/OJCS.2026.3664314.